

Smallest Multiple

Day 5 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“smallest number that is a multiple of every number”*
- ▶ *“Input consists of up to 500 test cases”*

Programming interpretation:

- ▶ For each line, compute the least common multiple of all listed values.
- ▶ The pairwise formula uses $\text{gcd}(a, b) * \text{lcm}(a, b) = a * b$.
- ▶ Fold the list from left to right until every number has been included.

Textbook connection: CP4 5.3.6 GCD and LCM

Core technique: Repeated LCM using GCD

- ▶ Each input line is an independent test case.
- ▶ The task is to find the smallest positive integer divisible by every number on the line.
- ▶ Input continues until EOF.

- ▶ The smallest number divisible by two integers is their LCM.
- ▶ The LCM operation can be folded across a list of numbers.
- ▶ GCD gives a safe and direct way to compute each pairwise LCM.

- ▶ Define $\text{lcm}(a, b) = (a * b) // \text{gcd}(a, b)$.
- ▶ For each nonempty input line, parse all integers.
- ▶ Set answer to the first number.
- ▶ For every remaining number x , set $\text{answer} = \text{lcm}(\text{answer}, x)$.
- ▶ Print the final answer.

- ▶ For 2 3 4, $\text{lcm}(2, 3) = 6$ and $\text{lcm}(6, 4) = 12$.
- ▶ Associativity lets us combine values one at a time.
- ▶ Python's `gcd` uses the Euclidean algorithm.

- ▶ At each step, answer is divisible by all processed numbers.
- ▶ The pairwise LCM is the smallest number that preserves divisibility by the previous answer and the new number.
- ▶ After all numbers are processed, answer is the smallest common multiple of the whole line.

Complexity: $O(K \log M)$ time per line for K numbers of size about M ; $O(K)$ memory to parse the line.

- ▶ In fixed-width languages, divide before multiplying to reduce overflow risk.
- ▶ Empty lines are skipped.

Source lines: `smallestmultiple.py:43-51`

```
import sys
from math import gcd

def lcm(a, b):
    return (a * b) // gcd(a, b)
```

What this chunk does

- ▶ Python's gcd gives the greatest common divisor efficiently.
- ▶ The helper rearranges the gcd-lcm identity to compute lcm(a, b).
- ▶ Integer division keeps the answer exact.

Source lines: `smallestmultiple.py:54-68`

```
def main():
    for line in sys.stdin:

        numbers = list(map(int, line.split()))

        if not numbers:
            continue

        answer = numbers[0]
```

What this chunk does

- ▶ Each input line is one independent test case.
- ▶ The loop over `sys.stdin` naturally stops at EOF.
- ▶ The first number initializes the running LCM.

Source lines: `smallestmultiple.py:69-82`

```
for i in range(1, len(numbers)):
    answer = lcm(answer, numbers[i])

print(answer)

main()
```

What this chunk does

- ▶ Each remaining number is combined with the current answer.
- ▶ After the loop, answer is divisible by every number on the line.
- ▶ The result is printed immediately for that test case.

Solution file: `practice_contests/day_5/smallestmultiple.py`

Problem: <https://open.kattis.com/problems/smallestmultiple>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.